📖 DAY 3: COMPLETE TOPIC-BY-TOPIC MASTER STUDY GUIDE

Deep Learning on HPC with TensorFlow, Slurm & GPUs

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 3 (Strictly 3 Hours / 180 Minutes)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

🧭 MODULE ROADMAP

- **Module 1**: Deep Learning Workflows & I/O Bottlenecks on HPC
- **Module 2**: Hardware-Aware ML Optimization Principles
- **Module 3**: Intel-Optimized TensorFlow & oneDNN Primitives
- **Module 4**: DL Accelerators: Intel GPU Max & Habana Gaudi
- **Module 5**: Slurm Workload Manager Architecture & Mechanics
- **Module 6**: Enterprise Slurm Batch Job Scripting
- **Module 7**: Distributed Training Paradigms: Parameter Server vs. Ring-AllReduce
- **Module 8**: Horovod Distributed Framework & Scaling Math

Module 1: Deep Learning Workflows & I/O Bottlenecks on HPC

1.1 The Machine Learning Life Cycle on Supercomputers

Traditional machine learning on a laptop typically processes small datasets that fit into local memory. In an enterprise HPC environment, deep learning workflows operate at petabyte scale:

1. **Raw Ingestion**: Petabytes of imagery, sensor data, or audio stored on parallel filesystems (Lustre / GPFS).
2. **Preprocessing**: Distributed tokenization and normalization utilizing multi-core Xeon CPUs.
3. **High-Throughput Staging**: Transferring active mini-batches to fast node-local `/scratch` NVMe storage.
4. **Distributed Execution**: Multi-node training across GPU/accelerator clusters orchestrated by Slurm.
5. **Model Checkpointing & Evaluation**: Atomic serialization of weights to persistent storage.

1.2 The I/O Starvation Bottleneck

A common failure in HPC deep learning is **GPU Starvation**: modern accelerators compute billions of operations per second, but if the CPU data pipeline cannot decode, augment, and transfer images over the PCIe bus fast enough, GPU utilization drops below 15%!

- **Remedies**:
  - Use `tf.data.Dataset.prefetch(tf.data.AUTOTUNE)` to overlap compute with I/O.

- Store datasets as binary records (`TFRecords` or Parquet) rather than millions of loose `.jpg` files.
- Utilize RAM-backed filesystems (`/dev/shm`) for small validation sets.

Module 2: Hardware-Aware ML Optimization Principles

2.1 Operator Fusion (Kernel Fusion)

In deep neural networks, a sequence of layers such as $\text{Output} = \text{ReLU}(\text{Conv2D}(X) + \text{Bias})$ traditionally requires three separate kernel launches:

1. Conv2D reads X from DRAM $\rightarrow$ Computes $\rightarrow$ Writes to DRAM.
2. Bias Add reads from DRAM $\rightarrow$ Adds bias $\rightarrow$ Writes to DRAM.
3. ReLU reads from DRAM $\rightarrow$ Clamps negatives to zero $\rightarrow$ Writes to DRAM.

Operator Fusion combines these operations into a single continuous hardware instruction stream. Data remains hot in CPU L1/L2 cache or GPU register files, eliminating two DRAM round-trips and delivering a **2× to 3× speedup**.

2.2 Tensor Memory Layouts (NHWC vs. NCHW)

- **NCHW (Channels First)**: Batch, Channels, Height, Width. Historically preferred by cuDNN on NVIDIA GPUs.
- **NHWC (Channels Last)**: Batch, Height, Width, Channels. Native format for Intel Xeon vector engines and oneDNN, allowing contiguous memory access across vector registers.

Module 3: Intel-Optimized TensorFlow & oneDNN Primitives

3.1 Intel oneDNN (oneAPI Deep Neural Network Library)

Intel oneDNN is an open-source performance library that optimizes deep learning computational primitives for Intel architecture:

- Auto-vectorized 2D/3D Convolutions, GEMM, Pooling, Normalization, and Activation.
- Eliminates generic Python/C++ interpreter overhead.

3.2 Intel AMX (Advanced Matrix Extensions)

Introduced in 4th Gen Intel Xeon Scalable processors (Sapphire Rapids), **Intel AMX** introduces hardware tile registers (`tmm0` – `tmm7`) and a Tile Matrix Multiply (TMUL) unit:

- Computes 1024 Bfloat16 operations or 2048 INT8 operations **per clock cycle**!
- Enables high-speed deep learning inference and fine-tuning directly on host CPUs without requiring discrete GPUs.

Module 4: DL Accelerators: Intel GPU Max & Habana Gaudi

4.1 Intel Data Center GPU Max 1550 (Ponte Vecchio)

- Up to 128 Xe-cores with hardware XMX (Xe Matrix Extensions) systolic engines.
- Up to 128 GB of HBM2e memory delivering over 3.2 TB/s of aggregate memory bandwidth.
- Linked via Intel XeLink bridges for coherent GPU-to-GPU memory sharing.

4.2 Habana Gaudi (Gaudi 2 & Gaudi 3) AI Accelerator

- Purpose-built ASIC for massive deep learning training and large language models.
- **Matrix Multiplication Engine (MME)**: Dedicated systolic array executing 3D tensor math.
- **Tensor Processor Cores (TPC)**: Custom VLIW SIMD cores for non-linear activations (Softmax, GELU, RoPE).
- **Integrated Scale-Out Networking**: 24 ports of 100/200 Gigabit RoCE v2 integrated directly onto the silicon die, eliminating external InfiniBand adapters!

Module 5: Slurm Workload Manager Architecture & Mechanics

5.1 Why Resource Managers are Mandatory

In a supercomputer, compute nodes are shared among hundreds of engineers. Users are **strictly prohibited** from running training jobs on the Head/Login node. Slurm acts as the cluster gatekeeper:

- `slurmctld`: The central controller daemon running on the head node. Evaluates resource requests and schedules jobs.
- `slurmd`: The worker daemon running on each compute node. Allocates cores, GPUs, and memory.

5.2 Key Slurm Commands

- `sbatch <script.slurm>`: Submits a non-interactive batch job to the queue.
- `squeue -u <user>`: Displays active jobs and queued priorities.
- `scancel <job_id>`: Terminates a running or queued job.
- `sinfo`: Queries partition states (e.g., available, allocated, drained nodes).

Module 6: Enterprise Slurm Batch Job Scripting

6.1 Structure of a Slurm Batch Script

```
#!/bin/bash
#SBATCH --job-name=resnet50_train
#SBATCH --output=/scratch/logs/%j.out
#SBATCH --error=/scratch/logs/%j.err
#SBATCH --partition=compute
#SBATCH --nodes=4
#SBATCH --ntasks-per-node=8
#SBATCH --cpus-per-task=4
#SBATCH --time=04:00:00

# Activate environments and run training
source /opt/intel/oneapi/setvars.sh
mpirun -n ${SLURM_NTASKS} python3 train.py
```

Module 7: Distributed Training Paradigms: Parameter Server vs. Ring-AllReduce

7.1 Parameter Server Architecture (Bottlenecked)

In early distributed deep learning, worker nodes computed gradients and sent them to a centralized **Parameter Server**. As worker counts scaled past 16 nodes, the Parameter Server network card saturated, creating a massive choke point ($O(N)$ communication overhead).

7.2 The Ring-AllReduce Algorithm (Bandwidth Optimal)

Horovod uses the **Ring-AllReduce algorithm** where workers are organized in a logical ring ($0 \to 1 \to 2 \cdots \to N-1 \to 0$):

1. **Scatter-Reduce**: Workers transmit small tensor chunks to neighbors for $N-1$ steps.
2. **AllGather**: Reduced tensor chunks circulate around the ring for another $N-1$ steps.

- Total communication volume is independent of node count N ($2 \times \frac{N-1}{N} \times \text{ModelSize}$), delivering near-linear scaling!

Module 8: Horovod Distributed Framework & Scaling Math

8.1 5 Core Code Changes for Horovod

```
import horovod.tensorflow as hvd
hvd.init()

# 1. Pin GPU to local rank
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    tf.config.set_visible_devices(gpus[hvd.local_rank()], 'GPU')

# 2. Linear Learning Rate Scaling Rule
scaled_lr = base_lr * hvd.size()
opt = tf.keras.optimizers.Adam(scaled_lr)

# 3. Wrap Optimizer for Ring-AllReduce
opt = hvd.DistributedOptimizer(opt)

# 4. Broadcast initial weights from Rank 0
hvd.broadcast_variables(model.variables, root_rank=0)
```

8.2 Scaling Metrics

- **Throughput (Images/sec)**: $\text{FPS} = \frac{\text{Global Batch Size} \times \text{Steps}}{\text{Elapsed Time}}$
- **Scaling Efficiency**: $\text{Efficiency} = \frac{\text{FPS}_{K \text{ workers}}}{K \times \text{FPS}_{1 \text{ worker}}} \times 100\%$

Study Guide Status: 100% Complete & Verified | GLA University HPC Masterclass Day 3